Experiment – 3

Development of an Interactive Event Page using React Hooks and State Management

---

Name :Jolin Saraiya

Class :D15A

Roll No. :45

Date :27OCTOBER

Aim

Develop an using **React Hooks and State Management**, and to understand the use of useState, useEffect, and useContext for managing component state, handling side effects, and sharing data between React components.

Problem Statement

Develop an interactive Event Management Page using React.

Enhance the Event Page developed in the previous experiment by adding interactive features such as **event search, category filtering, registration status, and shared application state**.

The application should demonstrate the use of React Hooks for managing and updating data dynamically.

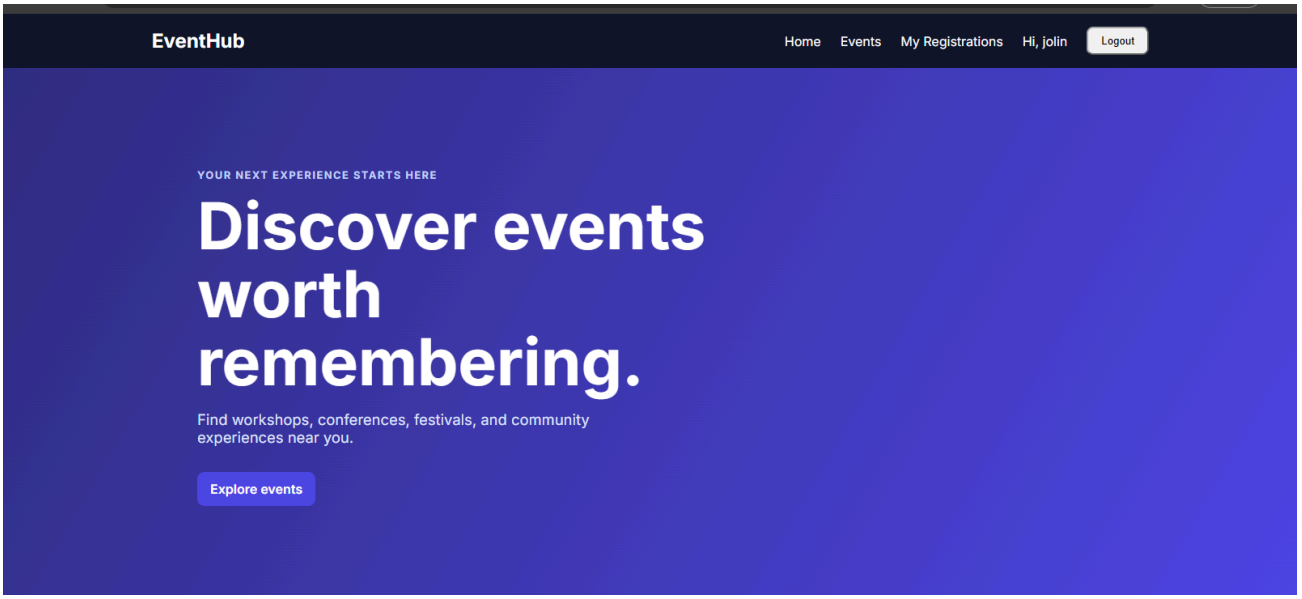
The following React concepts should be demonstrated:

- *useState* for managing local component state
- *useEffect* for handling side effects
- *useContext* for sharing state between components
- State management using Context API
- Event handling in React
- Conditional rendering
- Passing and updating state between components
- Dynamic rendering of event information

Or the same can be demonstrated on your project topic.

Implemented Features

- Dynamic event listing from the Express API.
- Search by event title, description, or location.
- Category filtering.
- User registration and login.
- Event registration with duplicate-registration protection.
- Persistent registration status loaded from the API.
- My Registrations page.
- Admin event creation and deletion.
- Responsive navigation menu.
- Loading, success, error, and empty-result states.



Theory Questions

1. What are React Hooks?

React Hooks are functions that allow function components to use React features such as state, lifecycle side effects, context, and memoization. Examples include useState, useEffect, and useContext.

2. What is the purpose of useState()?

useState() adds local state to a function component. It returns the current state value and a setter function. Calling the setter schedules a re-render with the updated value.

3. What is the difference between a normal variable and state in React?

A normal variable is reset when the component function runs again and changing it does not re-render the UI. React state persists between renders, and its setter informs React that the UI should be updated.

4. What is useEffect()?

useEffect() runs side-effect logic in a function component. Side effects include API calls, subscriptions, timers, browser storage updates, and document-title changes.

5. When is useEffect() executed?

An effect runs after the component renders. With an empty dependency array it runs after the initial mount. With dependencies it runs after mount and whenever one of those dependencies changes.

6. What is the purpose of the dependency array in useEffect()?

The dependency array controls when an effect runs. It prevents unnecessary work and ensures that the effect reacts to the values it uses. An empty array means the effect runs once after mounting.

7. What is useContext()?

useContext() reads a value from a React Context. It lets a component access shared data without receiving that data through every intermediate component as a prop.

8. What is Context API in React?

The Context API provides a way to create shared application data. A context provider supplies a value to descendants, and consumers or useContext() read that value.

9. What is prop drilling?

Prop drilling is the practice of passing data through several intermediate components even when those components do not use the data themselves, only so that a deeply nested component can receive it.

10. How does Context API help avoid prop drilling?

Context makes shared values available to components below a provider. Components can read the value directly, so every intermediate component does not need to forward the same props.

11. What is state management?

State management is the process of storing, reading, updating, and synchronizing data that controls an application's UI and behavior. State can be local to a component or shared across the application.

12. What is the difference between local state and global/shared state?

Local state belongs to one component and is useful for values such as an input or a menu toggle. Shared state is available to multiple components and is useful for authentication, user registrations, or application-wide preferences.

13. What is conditional rendering in React?

Conditional rendering displays different JSX based on a condition. This application uses it for loading indicators, errors, empty search results, authenticated navigation, and the registered-event message.

14. How does React update the UI when state changes?

When a state setter is called, React schedules a re-render. React compares the new element tree with the previous one and updates only the necessary DOM changes.

15. Why are Hooks useful in modern React applications?

Hooks allow function components to use state, effects, context, and reusable logic without class components. They make related behavior easier to organize and share through custom Hooks.

Important Code Snippets

Local state and event handling

```
const [search, setSearch] = useState("");const [category, setCategory] =
useState("all");<input value={search} onChange={(event) => setSearch(event.target.value)}
/>
```

Side effect for loading events

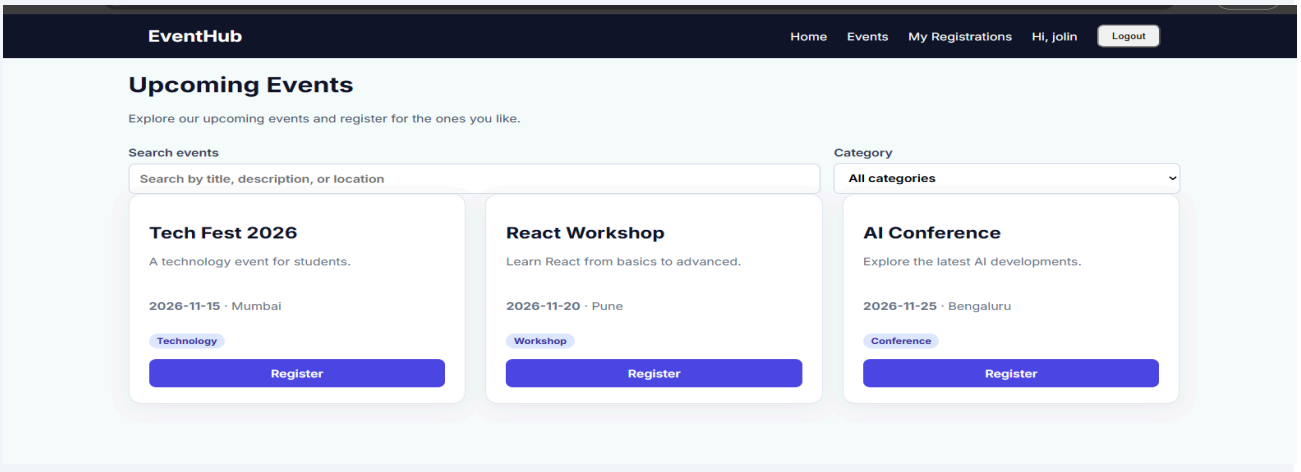
```
useEffect(() => {  async function loadEvents() {    try {      setEvents((await
api.get("/events")).data);    } finally {      setLoading(false);    }  }  loadEvents();},
[]);
```

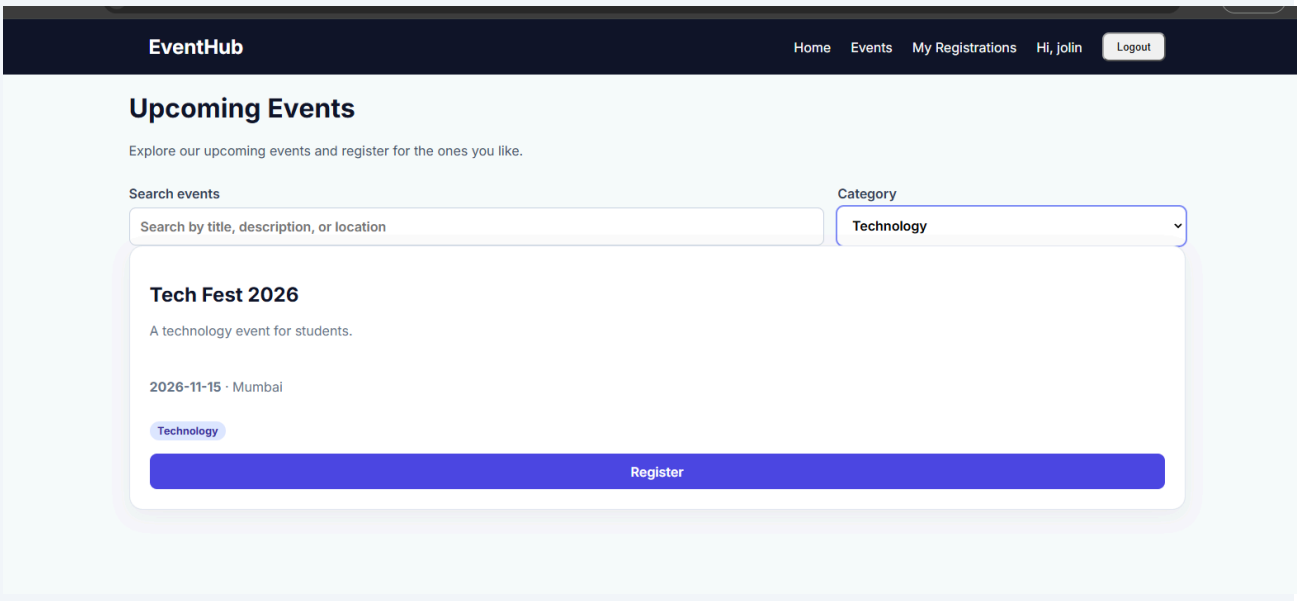
Context provider

```
export const AuthContext = createContext(null);export function AuthProvider({ children }) {
const [user, setUser] = useState(null);  const value = useMemo(() => ({ user, setUser }),
[user]);  return (    <AuthContext.Provider value={value}>      {children}
</AuthContext.Provider>  );}
```

Search and category filtering

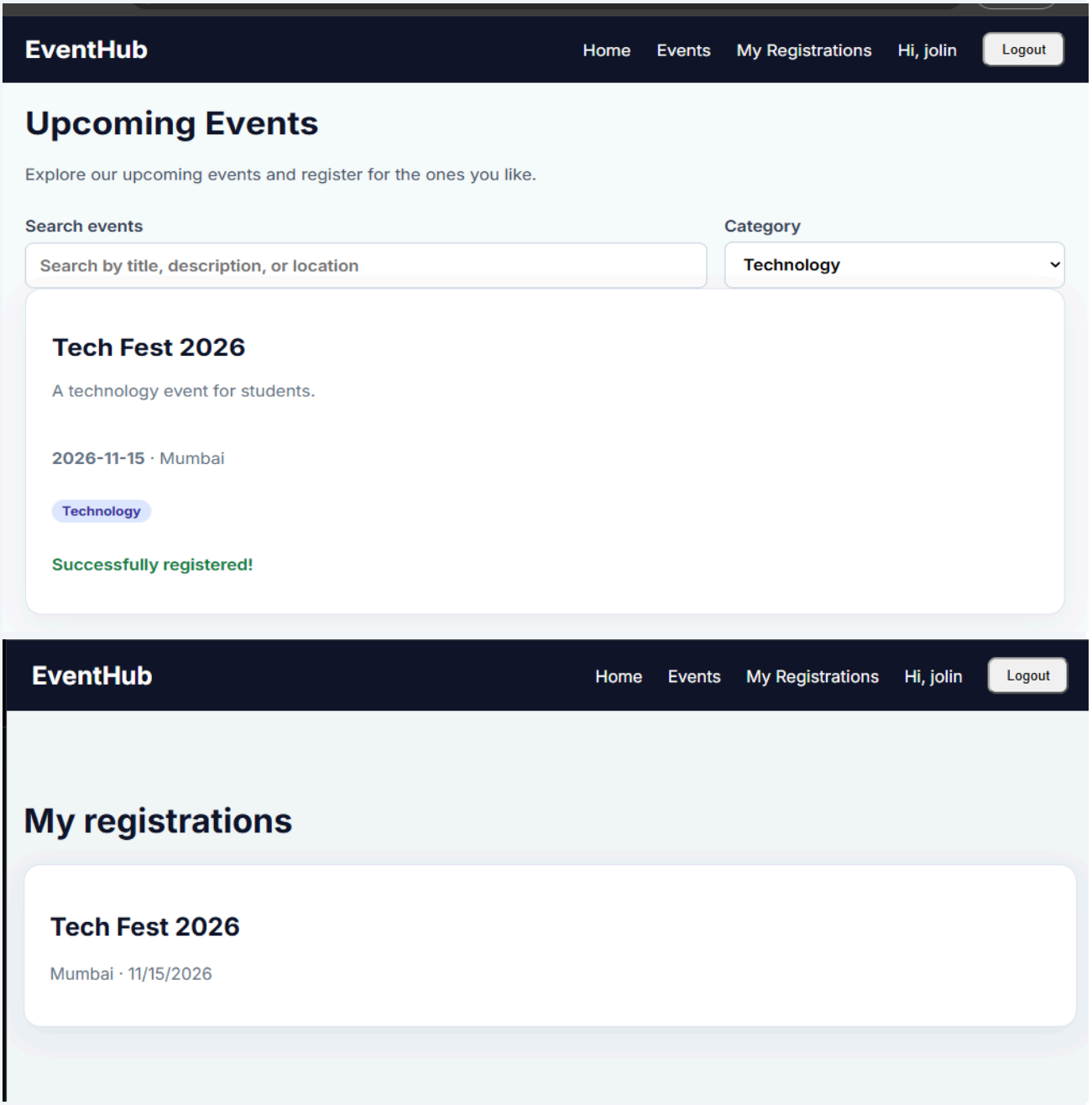
```
const filteredEvents = events.filter((event) => {  const query = search.trim().toLowerCase();
const matchesSearch = !query ||    [event.title, event.description, event.location]
.some((value) => value?.toLowerCase().includes(query));  const matchesCategory =    category ===
"all" || event.category === category;  return matchesSearch && matchesCategory;});
```





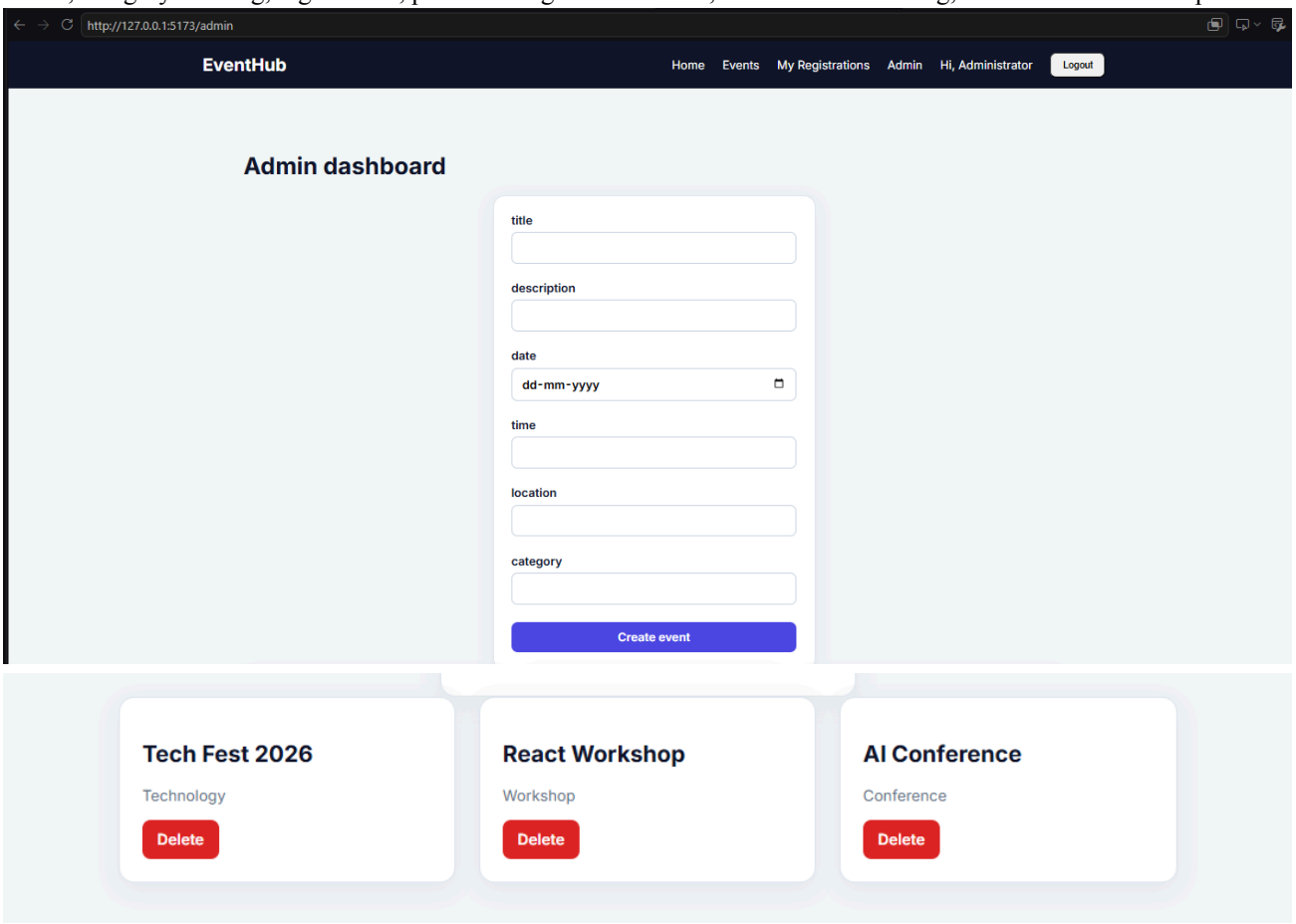
Event registration

```
async function register() {  try {    await api.post(`/registrations/${event._id}`);
setRegistered(true);    onRegistered?.(event._id);  } catch (error) {
setError(error.response?.data?.message || "Unable to register.");  }}
```



Result

The application successfully demonstrates React Hooks, Context API authentication state, dynamic event rendering, search, category filtering, registration, persistent registration status, conditional rendering, and API-based state updates.



Repository Link: https://github.com/JolinSaraiya/J-FSD.git